

CS184A/284A: AI in Biology and Medicine

HW3

Predicting TF Binding Sites

Transcription Factors (TFs) are proteins that bind to the DNA and help regulate gene transcription. The TFs have to recognize some "motif" on the DNA upstream from the gene, and DNA accessibility also plays a role.

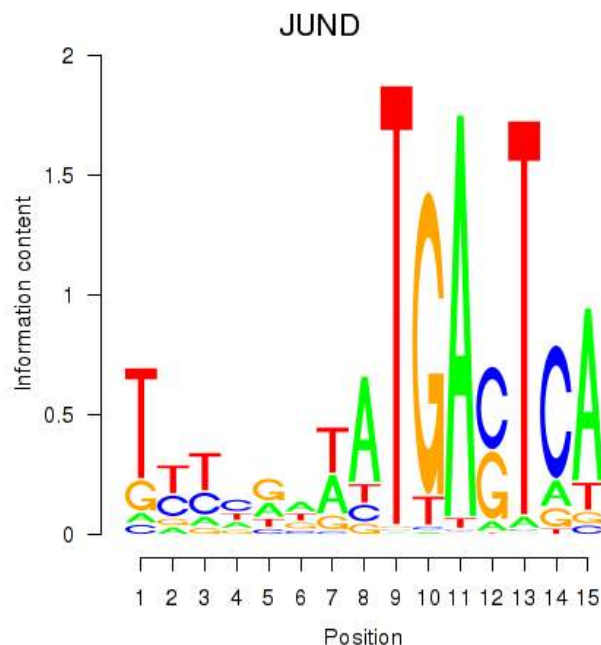
In this problem set, we will develop ML methods to predict which sequences can be bound by a transcription factor called JUND. The binding profile of JUND expressed in terms of a sequence logo is shown in the following picture.

More information on JUND can be found here:

- <https://www.genecards.org/cgi-bin/carddisp.pl?gene=JUND>

```
In [1]: # get the image
from IPython import display
display.Image("https://www.ismara.unibas.ch/ISMARA/scratch/NHBE_SC2/ismara_r
```

Out[1]:



MLP model

In this assignment you'll write an MLP model the predict whether a segments of the human chromosome 22 (Chr22) contain the binding sites for the JUND TF. You can modify the mlp notebook I shared with you to work on this problem. You need to have at least one hidden layer. You have to compute a weighted loss, and include accessibility information in your model, as described below.

Dataset

The data comprises 101 length segments from Chr22, with each position a one-hot vector denoting one of the four bases (A, C, G, T). Thus, each element of the input is 2d with size 101×4. Each such element has a target label 0 or 1, indicating whether the TF binds to that segment or not. The data also includes a weight per input element, since there are only a few binding sites (0.42%), so that you'd obtain an accuracy of 99.58% just by predicting there are no binding sites. This means you have to use the weights to discount the losses for label 0 and enhance the losses for label 1 items. Finally, there is an array of values, one per input element, that also indicates the chromosome accessibility for that segment.

Data Credit: Mohammed Zaki

Download data

- First, you need to download data file named "TF_data.zip" from Canvas. Unzip it and create the train, valid, test directories.
- If you use Google Colab, you can first upload the TF_data.zip file, and then run the follow command

```
!unzip TF_data.zip
```

- The data is split into training, validation and testing sets. Each set contains the following files:
 - shard-0-X.joblib: the set of 101 x 4 input elements
 - shard-0-y.joblib: the true labels: 0 or 1
 - shard-0-w.joblib: weight per input element
 - shard-0-a.joblib: accessibility value per input element
- After unzip the data file, you can read these files by using joblib.load function, which will populate a numpy array. For example

```
X = joblib.load('shard-0-X.joblib')
```

will results in a numpy array X, which you can then convert to torch tensor, and so on.

- The roles of training, validation and testing sets:

- Use training set to tune the parameters of the model.
- Use validation set to select model structure and hyperparameters (e.g., number of epochs, learning rate, etc).
- Use test set for the final evaluation. You should never touch test set for either your model training or model selection.

```
In [ ]: # Uncomment the following command if you run this code in Google Colab
#!unzip TF_data.zip
```

```
In [2]: import torch
import torch.nn as nn
import numpy as np
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import joblib
from sklearn.metrics import precision_score, recall_score, confusion_matrix,
```

Set up DataLoader for fetching training and testing data

Because we use mini-batch gradient descent for training. We need to set up dataloader that can provide us a minibatch of data samples.

```
In [3]: from torch._C import dtype
from torch.utils.data import Dataset

class JUND_Dataset(Dataset):
    def __init__(self, data_dir):
        '''load X, y, w, a from data_dir'''
        super(JUND_Dataset, self).__init__()

        # load X, y, w, a from given data_dir
        # convert them into torch tensors
        self.X = torch.from_numpy(joblib.load(data_dir + '/shard-0-X.joblib')
        self.y = torch.from_numpy(joblib.load(data_dir + '/shard-0-y.joblib')
        self.w = torch.from_numpy(joblib.load(data_dir + '/shard-0-w.joblib')
        self.a = torch.from_numpy(joblib.load(data_dir + '/shard-0-a.joblib')

    def __len__(self):
        '''return len of dataset'''
        return self.X.shape[0]

    def __getitem__(self, idx):
        '''return X, y, w, and a values at index idx'''
        return self.X[idx], self.y[idx], self.w[idx], self.a[idx]
```

```
In [4]: # get data
#
# You may need to change the directory if the data are not stored under curr
```

```
#  
train_dataset = JUND_Dataset('train_dataset')  
valid_dataset = JUND_Dataset('valid_dataset')  
test_dataset = JUND_Dataset('test_dataset')
```

Training and test data

```
In [5]: batch_size = 100  
  
# Data loader  
train_loader = torch.utils.data.DataLoader(dataset=train_dataset,  
                                             batch_size=batch_size,  
                                             shuffle=True)  
  
valid_loader = torch.utils.data.DataLoader(dataset=valid_dataset,  
                                             batch_size=batch_size,  
                                             shuffle=False)  
  
test_loader = torch.utils.data.DataLoader(dataset=test_dataset,  
                                           batch_size=batch_size,  
                                           shuffle=False)
```

```
In [6]: print('Train data: ', len(train_dataset))  
        print('Test data: ', len(test_dataset))
```

Train data: 276216

Test data: 34528

Your result should look like:

Train data: 276216

Test data: 34528

Fetch a minibatch and check the size of the data

```
In [7]: X,y,w,a = next(iter(train_loader))
```

```
In [8]: # run the following code to check the size of data in each minibatch  
        X.shape, y.shape, w.shape, a.shape
```

```
Out[8]: (torch.Size([100, 101, 4]),  
        torch.Size([100, 1]),  
        torch.Size([100, 1]),  
        torch.Size([100, 1]))
```

Your result should look like the following:

```
(torch.Size([100, 101, 4]),  
 torch.Size([100, 1]),  
 torch.Size([100, 1]),  
 torch.Size([100, 1]))
```

Problem 1 - MLP

Define an MLP (multi-layer perceptron) with at least one hidden layer to predict the labels given inputs.

Please note the following:

- The label for each input is either 0 or 1, so this is essentially a binary classification problem.
- Input consist of both X and a:
 - X: represents the DNA sequence. Each position is a one-hot vector denoting one of the four bases (A, C, G, T). Thus, each element of the input is 2d with size 101×4. Since each input is treated as a vector in MLP, the 2d array needs to be flattened into a 404-dimensional vector.
 - a: represents the chromatin accessibility of the input DNA sequence segment. You can think of "a" as an additional feature for each input. You can decide how to use it. For instance, if you are using hidden dimension of 128, then after concatenating the accessibility value, it will become a 129d vector, which should be fed to the final output layer of size 1, since we have a binary class/label.

An initial template code, representing a simple model, is provided. Your job is to change the definition of the model to improve the model's performance on the test dataset.

```
In [9]: #####
# modify this code block
#####
# MLP
class MyModel(nn.Module):
    def __init__(self, input_size=101*4, hidden_size=128, hidden_size2=64):
        super(MyModel, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Dropout(0.3))
        self.layer2 = nn.Sequential(
            nn.Linear(hidden_size, hidden_size2),
            nn.ReLU(),
            nn.Dropout(0.3))
        self.fc = nn.Linear(hidden_size2+1, 1)

    def forward(self, X, a):
        out = X.reshape(X.size(0),-1)
        out = self.layer1(out)
        out = self.layer2(out)
        out = torch.cat((out, a),1)
        out = self.fc(out)
        return out
```

```
# end of model definition
#####
```

Weight Accuracy Function

First, we define the weighted accuracy function, then the evaluation function.

```
In [10]: #####
# Weight Accuracy Function
#####
def weight_accuracy(predicted_y, true_y, weight):
    """
    Inputs:
        predicted_y: predicted labels
        true_y: true labels
        weight: weight of each sample
    return:
        sum of the weights of the correct predictions divided by the total w
    """
    # Convert inputs to numpy arrays if they are tensors
    if isinstance(predicted_y, torch.Tensor):
        predicted_y = predicted_y.cpu().numpy()
    if isinstance(true_y, torch.Tensor):
        true_y = true_y.cpu().numpy()
    if isinstance(weight, torch.Tensor):
        weight = weight.cpu().numpy()

    # Flatten arrays if needed
    predicted_y = np.array(predicted_y).flatten()
    true_y = np.array(true_y).flatten()
    weight = np.array(weight).flatten()

    # Find correct predictions
    correct = (predicted_y == true_y)

    # Sum of weights for correct predictions
    correct_weight_sum = np.sum(weight[correct])

    # Total weight
    total_weight = np.sum(weight)

    # Weighted accuracy
    weighted_acc = correct_weight_sum / total_weight if total_weight > 0 else

    return weighted_acc
#####

def evaluate_model(model, data_loader):
    """Evaluate model and return weighted accuracy"""
    model.eval()
    all_predictions = []
    all_labels = []
    all_weights = []
```

```

with torch.no_grad():
    for X, y, w, a in data_loader:
        output = model(X, a)
        y_pred_tag = (output > 0).int()
        all_predictions.append(y_pred_tag.cpu().numpy())
        all_labels.append(y.cpu().numpy())
        all_weights.append(w.cpu().numpy())

    # Flatten all lists
    all_predictions = np.concatenate([arr.flatten() for arr in all_predictions])
    all_labels = np.concatenate([arr.flatten() for arr in all_labels])
    all_weights = np.concatenate([arr.flatten() for arr in all_weights])

    return weight_accuracy(all_predictions, all_labels, all_weights)

```

Have a test run of your model

```

In [11]: model = MyModel()
         output = model(X,a)
         print(f"Output shape: {output.shape}")

```

Output shape: torch.Size([100, 1])

Your model should run smoothly. The size of output should be 100 - the same as the minibatch size.

Training

Next you need to define a loss function and then run gradient descend to learn the weights of the neural net.

- There is a strong class imbalance problem in the training set (many more 0's than 1's).
- To handle the class imbalance problem, we treat each sample differently. Each data point is assigned a weight. Take a look at the variable named "w" in each data set.
- Define a loss function, in which the total loss is a weighted combination of losses coming from each sample. Use the weights specified in "w". Note that this definition is different from our typical loss, where each sample contributes equally to the final total loss.
- You should use `binary_cross_entropy_with_logits` with weight set to the weights per input element. Check out the documentation for details.

```

In [12]: #####
         # complete this code block
         # define loss and optimizer
         #####

```

```
# Note: We'll set the weight per sample in the training loop
# BCEWithLogitsLoss with reduction='none' allows us to apply weights per sample
criterion = nn.BCEWithLogitsLoss(reduction='none')
#####
```

```
In [13]: # have a test run to check if your loss is defined properly
# First test without weights
loss_unweighted = criterion(output, y)
print(f"Unweighted loss shape: {loss_unweighted.shape}")
# Now test with weights
weighted_loss = (loss_unweighted * w.squeeze()).mean()
print(f"Weighted loss: {weighted_loss.item()}")
```

```
Unweighted loss shape: torch.Size([100, 1])
Weighted loss: 1.157656192779541
```

```
In [ ]:
```

You task:

You need to train the model on the training data, and use the **validation data** to select how many epochs you want to use and to choose the hidden dimension.

Use the **weighted prediction accuracy** as the evaluation metric. That is, sum of the weights of the correct predictions divided by the total weight across all the input elements.

Finally, report the **weighted accuracy on the test data**.

Run Training

```
In [14]: # Choose hyper parameters and optimizer
num_epochs = 10
learning_rate = 0.001
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

```
In [15]: # Train the model
total_step = len(train_loader)
model.train()
for epoch in range(num_epochs):
    for i, (X, y, w, a) in enumerate(train_loader):
        # Forward pass
        output = model(X, a)
        # Compute unweighted loss per sample
        loss_per_sample = criterion(output, y)
        # Apply weights and compute mean
        loss = (loss_per_sample * w.squeeze()).mean()

        # Backward and optimize
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```
if (i+1) % 100 == 0:
    print ('Epoch [{}/{}], Step [{}/{}], Loss: {:.4f}'
          .format(epoch+1, num_epochs, i+1, total_step, loss.item()))

# Evaluate on validation set after each epoch
valid_acc = evaluate_model(model, valid_loader)
print(f'Epoch [{epoch+1}/{num_epochs}], Validation Weighted Accuracy: {v
```

Epoch [1/10], Step [100/2763], Loss: 0.0053
Epoch [1/10], Step [200/2763], Loss: 0.0042
Epoch [1/10], Step [300/2763], Loss: 0.0694
Epoch [1/10], Step [400/2763], Loss: 0.0911
Epoch [1/10], Step [500/2763], Loss: 0.1487
Epoch [1/10], Step [600/2763], Loss: 0.0056
Epoch [1/10], Step [700/2763], Loss: 0.1116
Epoch [1/10], Step [800/2763], Loss: 0.0049
Epoch [1/10], Step [900/2763], Loss: 0.0033
Epoch [1/10], Step [1000/2763], Loss: 0.0974
Epoch [1/10], Step [1100/2763], Loss: 0.0911
Epoch [1/10], Step [1200/2763], Loss: 0.1170
Epoch [1/10], Step [1300/2763], Loss: 0.0045
Epoch [1/10], Step [1400/2763], Loss: 0.0923
Epoch [1/10], Step [1500/2763], Loss: 0.0904
Epoch [1/10], Step [1600/2763], Loss: 0.0055
Epoch [1/10], Step [1700/2763], Loss: 0.0045
Epoch [1/10], Step [1800/2763], Loss: 0.0048
Epoch [1/10], Step [1900/2763], Loss: 0.0031
Epoch [1/10], Step [2000/2763], Loss: 0.0897
Epoch [1/10], Step [2100/2763], Loss: 0.0087
Epoch [1/10], Step [2200/2763], Loss: 0.0126
Epoch [1/10], Step [2300/2763], Loss: 0.0136
Epoch [1/10], Step [2400/2763], Loss: 0.0099
Epoch [1/10], Step [2500/2763], Loss: 0.0063
Epoch [1/10], Step [2600/2763], Loss: 0.0047
Epoch [1/10], Step [2700/2763], Loss: 0.0030
Epoch [1/10], Validation Weighted Accuracy: 0.5000
Epoch [2/10], Step [100/2763], Loss: 0.0030
Epoch [2/10], Step [200/2763], Loss: 0.0042
Epoch [2/10], Step [300/2763], Loss: 0.0040
Epoch [2/10], Step [400/2763], Loss: 0.0041
Epoch [2/10], Step [500/2763], Loss: 0.0030
Epoch [2/10], Step [600/2763], Loss: 0.0029
Epoch [2/10], Step [700/2763], Loss: 0.0966
Epoch [2/10], Step [800/2763], Loss: 0.0988
Epoch [2/10], Step [900/2763], Loss: 0.0043
Epoch [2/10], Step [1000/2763], Loss: 0.3079
Epoch [2/10], Step [1100/2763], Loss: 0.0873
Epoch [2/10], Step [1200/2763], Loss: 0.0944
Epoch [2/10], Step [1300/2763], Loss: 0.1065
Epoch [2/10], Step [1400/2763], Loss: 0.0057
Epoch [2/10], Step [1500/2763], Loss: 0.0048
Epoch [2/10], Step [1600/2763], Loss: 0.0076
Epoch [2/10], Step [1700/2763], Loss: 0.1025
Epoch [2/10], Step [1800/2763], Loss: 0.0067
Epoch [2/10], Step [1900/2763], Loss: 0.0938
Epoch [2/10], Step [2000/2763], Loss: 0.0044
Epoch [2/10], Step [2100/2763], Loss: 0.3045
Epoch [2/10], Step [2200/2763], Loss: 0.0965
Epoch [2/10], Step [2300/2763], Loss: 0.1064
Epoch [2/10], Step [2400/2763], Loss: 0.0024
Epoch [2/10], Step [2500/2763], Loss: 0.0031
Epoch [2/10], Step [2600/2763], Loss: 0.0831
Epoch [2/10], Step [2700/2763], Loss: 0.0038
Epoch [2/10], Validation Weighted Accuracy: 0.5000

Epoch [3/10], Step [100/2763], Loss: 0.0996
Epoch [3/10], Step [200/2763], Loss: 0.0047
Epoch [3/10], Step [300/2763], Loss: 0.0031
Epoch [3/10], Step [400/2763], Loss: 0.0031
Epoch [3/10], Step [500/2763], Loss: 0.0039
Epoch [3/10], Step [600/2763], Loss: 0.0044
Epoch [3/10], Step [700/2763], Loss: 0.0918
Epoch [3/10], Step [800/2763], Loss: 0.0054
Epoch [3/10], Step [900/2763], Loss: 0.1008
Epoch [3/10], Step [1000/2763], Loss: 0.0040
Epoch [3/10], Step [1100/2763], Loss: 0.0832
Epoch [3/10], Step [1200/2763], Loss: 0.0066
Epoch [3/10], Step [1300/2763], Loss: 0.0928
Epoch [3/10], Step [1400/2763], Loss: 0.0038
Epoch [3/10], Step [1500/2763], Loss: 0.0878
Epoch [3/10], Step [1600/2763], Loss: 0.0021
Epoch [3/10], Step [1700/2763], Loss: 0.0099
Epoch [3/10], Step [1800/2763], Loss: 0.0054
Epoch [3/10], Step [1900/2763], Loss: 0.0029
Epoch [3/10], Step [2000/2763], Loss: 0.0068
Epoch [3/10], Step [2100/2763], Loss: 0.0030
Epoch [3/10], Step [2200/2763], Loss: 0.0050
Epoch [3/10], Step [2300/2763], Loss: 0.0047
Epoch [3/10], Step [2400/2763], Loss: 0.0058
Epoch [3/10], Step [2500/2763], Loss: 0.0029
Epoch [3/10], Step [2600/2763], Loss: 0.0036
Epoch [3/10], Step [2700/2763], Loss: 0.0044
Epoch [3/10], Validation Weighted Accuracy: 0.5000
Epoch [4/10], Step [100/2763], Loss: 0.0050
Epoch [4/10], Step [200/2763], Loss: 0.0026
Epoch [4/10], Step [300/2763], Loss: 0.0899
Epoch [4/10], Step [400/2763], Loss: 0.0045
Epoch [4/10], Step [500/2763], Loss: 0.0060
Epoch [4/10], Step [600/2763], Loss: 0.0047
Epoch [4/10], Step [700/2763], Loss: 0.0057
Epoch [4/10], Step [800/2763], Loss: 0.0885
Epoch [4/10], Step [900/2763], Loss: 0.0069
Epoch [4/10], Step [1000/2763], Loss: 0.0934
Epoch [4/10], Step [1100/2763], Loss: 0.2852
Epoch [4/10], Step [1200/2763], Loss: 0.0022
Epoch [4/10], Step [1300/2763], Loss: 0.0040
Epoch [4/10], Step [1400/2763], Loss: 0.0027
Epoch [4/10], Step [1500/2763], Loss: 0.0035
Epoch [4/10], Step [1600/2763], Loss: 0.0051
Epoch [4/10], Step [1700/2763], Loss: 0.0041
Epoch [4/10], Step [1800/2763], Loss: 0.0035
Epoch [4/10], Step [1900/2763], Loss: 0.0944
Epoch [4/10], Step [2000/2763], Loss: 0.0081
Epoch [4/10], Step [2100/2763], Loss: 0.0040
Epoch [4/10], Step [2200/2763], Loss: 0.0054
Epoch [4/10], Step [2300/2763], Loss: 0.0037
Epoch [4/10], Step [2400/2763], Loss: 0.0962
Epoch [4/10], Step [2500/2763], Loss: 0.0041
Epoch [4/10], Step [2600/2763], Loss: 0.0038
Epoch [4/10], Step [2700/2763], Loss: 0.0033
Epoch [4/10], Validation Weighted Accuracy: 0.5000

Epoch [5/10], Step [100/2763], Loss: 0.0027
Epoch [5/10], Step [200/2763], Loss: 0.0058
Epoch [5/10], Step [300/2763], Loss: 0.0112
Epoch [5/10], Step [400/2763], Loss: 0.0865
Epoch [5/10], Step [500/2763], Loss: 0.0051
Epoch [5/10], Step [600/2763], Loss: 0.2567
Epoch [5/10], Step [700/2763], Loss: 0.0025
Epoch [5/10], Step [800/2763], Loss: 0.0058
Epoch [5/10], Step [900/2763], Loss: 0.0038
Epoch [5/10], Step [1000/2763], Loss: 0.0031
Epoch [5/10], Step [1100/2763], Loss: 0.0057
Epoch [5/10], Step [1200/2763], Loss: 0.0030
Epoch [5/10], Step [1300/2763], Loss: 0.0797
Epoch [5/10], Step [1400/2763], Loss: 0.0034
Epoch [5/10], Step [1500/2763], Loss: 0.0026
Epoch [5/10], Step [1600/2763], Loss: 0.0040
Epoch [5/10], Step [1700/2763], Loss: 0.0067
Epoch [5/10], Step [1800/2763], Loss: 0.0026
Epoch [5/10], Step [1900/2763], Loss: 0.0046
Epoch [5/10], Step [2000/2763], Loss: 0.0030
Epoch [5/10], Step [2100/2763], Loss: 0.0045
Epoch [5/10], Step [2200/2763], Loss: 0.0033
Epoch [5/10], Step [2300/2763], Loss: 0.0982
Epoch [5/10], Step [2400/2763], Loss: 0.0070
Epoch [5/10], Step [2500/2763], Loss: 0.0030
Epoch [5/10], Step [2600/2763], Loss: 0.1037
Epoch [5/10], Step [2700/2763], Loss: 0.0043
Epoch [5/10], Validation Weighted Accuracy: 0.5000
Epoch [6/10], Step [100/2763], Loss: 0.0026
Epoch [6/10], Step [200/2763], Loss: 0.0038
Epoch [6/10], Step [300/2763], Loss: 0.0920
Epoch [6/10], Step [400/2763], Loss: 0.0579
Epoch [6/10], Step [500/2763], Loss: 0.0038
Epoch [6/10], Step [600/2763], Loss: 0.0018
Epoch [6/10], Step [700/2763], Loss: 0.0037
Epoch [6/10], Step [800/2763], Loss: 0.0034
Epoch [6/10], Step [900/2763], Loss: 0.0031
Epoch [6/10], Step [1000/2763], Loss: 0.0047
Epoch [6/10], Step [1100/2763], Loss: 0.0051
Epoch [6/10], Step [1200/2763], Loss: 0.0044
Epoch [6/10], Step [1300/2763], Loss: 0.0774
Epoch [6/10], Step [1400/2763], Loss: 0.0048
Epoch [6/10], Step [1500/2763], Loss: 0.0778
Epoch [6/10], Step [1600/2763], Loss: 0.0047
Epoch [6/10], Step [1700/2763], Loss: 0.0099
Epoch [6/10], Step [1800/2763], Loss: 0.0065
Epoch [6/10], Step [1900/2763], Loss: 0.0049
Epoch [6/10], Step [2000/2763], Loss: 0.0041
Epoch [6/10], Step [2100/2763], Loss: 0.0039
Epoch [6/10], Step [2200/2763], Loss: 0.0389
Epoch [6/10], Step [2300/2763], Loss: 0.0294
Epoch [6/10], Step [2400/2763], Loss: 0.0032
Epoch [6/10], Step [2500/2763], Loss: 0.0051
Epoch [6/10], Step [2600/2763], Loss: 0.0053
Epoch [6/10], Step [2700/2763], Loss: 0.0097
Epoch [6/10], Validation Weighted Accuracy: 0.5000

Epoch [7/10], Step [100/2763], Loss: 0.0023
Epoch [7/10], Step [200/2763], Loss: 0.0048
Epoch [7/10], Step [300/2763], Loss: 0.0028
Epoch [7/10], Step [400/2763], Loss: 0.0056
Epoch [7/10], Step [500/2763], Loss: 0.0811
Epoch [7/10], Step [600/2763], Loss: 0.0029
Epoch [7/10], Step [700/2763], Loss: 0.0038
Epoch [7/10], Step [800/2763], Loss: 0.0039
Epoch [7/10], Step [900/2763], Loss: 0.1081
Epoch [7/10], Step [1000/2763], Loss: 0.0013
Epoch [7/10], Step [1100/2763], Loss: 0.0818
Epoch [7/10], Step [1200/2763], Loss: 0.0028
Epoch [7/10], Step [1300/2763], Loss: 0.0033
Epoch [7/10], Step [1400/2763], Loss: 0.0022
Epoch [7/10], Step [1500/2763], Loss: 0.0414
Epoch [7/10], Step [1600/2763], Loss: 0.0047
Epoch [7/10], Step [1700/2763], Loss: 0.0173
Epoch [7/10], Step [1800/2763], Loss: 0.0756
Epoch [7/10], Step [1900/2763], Loss: 0.0741
Epoch [7/10], Step [2000/2763], Loss: 0.0057
Epoch [7/10], Step [2100/2763], Loss: 0.0025
Epoch [7/10], Step [2200/2763], Loss: 0.0030
Epoch [7/10], Step [2300/2763], Loss: 0.0034
Epoch [7/10], Step [2400/2763], Loss: 0.0652
Epoch [7/10], Step [2500/2763], Loss: 0.0044
Epoch [7/10], Step [2600/2763], Loss: 0.0778
Epoch [7/10], Step [2700/2763], Loss: 0.0026
Epoch [7/10], Validation Weighted Accuracy: 0.4998
Epoch [8/10], Step [100/2763], Loss: 0.1234
Epoch [8/10], Step [200/2763], Loss: 0.0013
Epoch [8/10], Step [300/2763], Loss: 0.0980
Epoch [8/10], Step [400/2763], Loss: 0.0062
Epoch [8/10], Step [500/2763], Loss: 0.0034
Epoch [8/10], Step [600/2763], Loss: 0.0014
Epoch [8/10], Step [700/2763], Loss: 0.0153
Epoch [8/10], Step [800/2763], Loss: 0.1522
Epoch [8/10], Step [900/2763], Loss: 0.0231
Epoch [8/10], Step [1000/2763], Loss: 0.0005
Epoch [8/10], Step [1100/2763], Loss: 0.0021
Epoch [8/10], Step [1200/2763], Loss: 0.0015
Epoch [8/10], Step [1300/2763], Loss: 0.0169
Epoch [8/10], Step [1400/2763], Loss: 0.0010
Epoch [8/10], Step [1500/2763], Loss: 0.0034
Epoch [8/10], Step [1600/2763], Loss: 0.0132
Epoch [8/10], Step [1700/2763], Loss: 0.0051
Epoch [8/10], Step [1800/2763], Loss: 0.0172
Epoch [8/10], Step [1900/2763], Loss: 0.0008
Epoch [8/10], Step [2000/2763], Loss: 0.0028
Epoch [8/10], Step [2100/2763], Loss: 0.0017
Epoch [8/10], Step [2200/2763], Loss: 0.4394
Epoch [8/10], Step [2300/2763], Loss: 0.0063
Epoch [8/10], Step [2400/2763], Loss: 0.0010
Epoch [8/10], Step [2500/2763], Loss: 0.0501
Epoch [8/10], Step [2600/2763], Loss: 0.0007
Epoch [8/10], Step [2700/2763], Loss: 0.0344
Epoch [8/10], Validation Weighted Accuracy: 0.4991

Epoch [9/10], Step [100/2763], Loss: 0.0094
Epoch [9/10], Step [200/2763], Loss: 0.0040
Epoch [9/10], Step [300/2763], Loss: 0.0101
Epoch [9/10], Step [400/2763], Loss: 0.0011
Epoch [9/10], Step [500/2763], Loss: 0.0021
Epoch [9/10], Step [600/2763], Loss: 0.0079
Epoch [9/10], Step [700/2763], Loss: 0.0019
Epoch [9/10], Step [800/2763], Loss: 0.0018
Epoch [9/10], Step [900/2763], Loss: 0.0102
Epoch [9/10], Step [1000/2763], Loss: 0.0004
Epoch [9/10], Step [1100/2763], Loss: 0.0178
Epoch [9/10], Step [1200/2763], Loss: 0.0145
Epoch [9/10], Step [1300/2763], Loss: 0.0015
Epoch [9/10], Step [1400/2763], Loss: 0.0054
Epoch [9/10], Step [1500/2763], Loss: 0.0012
Epoch [9/10], Step [1600/2763], Loss: 0.0006
Epoch [9/10], Step [1700/2763], Loss: 0.0339
Epoch [9/10], Step [1800/2763], Loss: 0.0051
Epoch [9/10], Step [1900/2763], Loss: 0.0032
Epoch [9/10], Step [2000/2763], Loss: 0.0025
Epoch [9/10], Step [2100/2763], Loss: 0.0008
Epoch [9/10], Step [2200/2763], Loss: 0.0443
Epoch [9/10], Step [2300/2763], Loss: 0.0022
Epoch [9/10], Step [2400/2763], Loss: 0.0010
Epoch [9/10], Step [2500/2763], Loss: 0.1732
Epoch [9/10], Step [2600/2763], Loss: 0.0111
Epoch [9/10], Step [2700/2763], Loss: 0.0386
Epoch [9/10], Validation Weighted Accuracy: 0.4987
Epoch [10/10], Step [100/2763], Loss: 0.0018
Epoch [10/10], Step [200/2763], Loss: 0.0035
Epoch [10/10], Step [300/2763], Loss: 0.0005
Epoch [10/10], Step [400/2763], Loss: 0.0027
Epoch [10/10], Step [500/2763], Loss: 0.0021
Epoch [10/10], Step [600/2763], Loss: 0.0008
Epoch [10/10], Step [700/2763], Loss: 0.0017
Epoch [10/10], Step [800/2763], Loss: 0.0005
Epoch [10/10], Step [900/2763], Loss: 0.0117
Epoch [10/10], Step [1000/2763], Loss: 0.0032
Epoch [10/10], Step [1100/2763], Loss: 0.0321
Epoch [10/10], Step [1200/2763], Loss: 0.0032
Epoch [10/10], Step [1300/2763], Loss: 0.0012
Epoch [10/10], Step [1400/2763], Loss: 0.0011
Epoch [10/10], Step [1500/2763], Loss: 0.0142
Epoch [10/10], Step [1600/2763], Loss: 0.0006
Epoch [10/10], Step [1700/2763], Loss: 0.0354
Epoch [10/10], Step [1800/2763], Loss: 0.0007
Epoch [10/10], Step [1900/2763], Loss: 0.0055
Epoch [10/10], Step [2000/2763], Loss: 0.0217
Epoch [10/10], Step [2100/2763], Loss: 0.0011
Epoch [10/10], Step [2200/2763], Loss: 0.0141
Epoch [10/10], Step [2300/2763], Loss: 0.0556
Epoch [10/10], Step [2400/2763], Loss: 0.0008
Epoch [10/10], Step [2500/2763], Loss: 0.0009
Epoch [10/10], Step [2600/2763], Loss: 0.0031
Epoch [10/10], Step [2700/2763], Loss: 0.0010
Epoch [10/10], Validation Weighted Accuracy: 0.4996

Final Evaluation

Generate predictions on test set

```
In [16]: import itertools

y_pred_list = []
y_target_list = []
model.eval()
#Since we don't need model to back propagate the gradients in test set we use eval mode
# reduces memory usage and speeds up computation
with torch.no_grad():
    for i, (X, y, w, a) in enumerate(test_loader):
        output = model(X, a)
        y_pred_tag = (output>0).int()
        y_pred_list.append(y_pred_tag.detach().numpy())
        y_target_list.append(y.detach().numpy())

#Takes arrays and makes them list of list for each batch
y_pred_list = [a.squeeze().tolist() for a in y_pred_list]
#flattens the lists in sequence
ytest_pred = list(itertools.chain.from_iterable(y_pred_list))

#Takes arrays and makes them list of list for each batch
y_target_list = [a.squeeze().tolist() for a in y_target_list]
#flattens the lists in sequence
ytest_target = list(itertools.chain.from_iterable(y_target_list))
```

Report

- Precision
- Recall
- F1 Score
- Confusion Matrix

```
In [19]: conf_matrix = confusion_matrix(ytest_target, ytest_pred)
print("Confusion Matrix of the Test Set")
# Visualize confusion matrix with scikit-learn
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

fig, ax = plt.subplots(figsize=(6, 6))
ConfusionMatrixDisplay.from_predictions(
    ytest_target,
    ytest_pred,
    cmap='Blues',          # color map
    normalize=None,        # raw counts; use 'true' for recall-normalized
    # display_labels=class_names, # optional: pass your class names here
    ax=ax
)
```

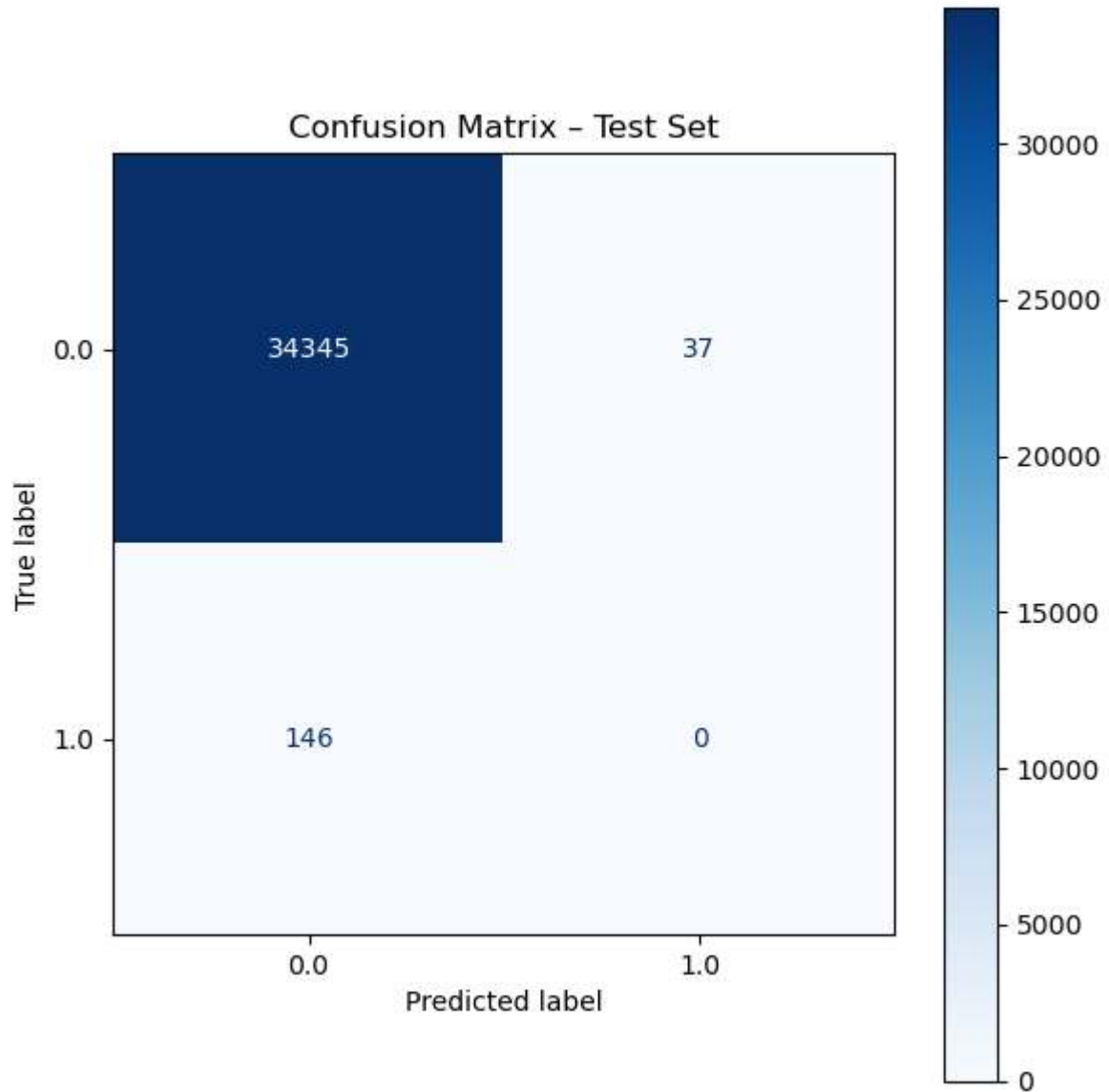
```

ax.set_title('Confusion Matrix - Test Set')
plt.tight_layout()
plt.show()

print("-----")
print(conf_matrix)
print("Precision of the MLP :\t"+str(precision_score(ytest_target,ytest_pred))
print("Recall of the MLP    :\t"+str(recall_score(ytest_target,ytest_pred)))
print("F1 Score of the Model :\t"+str(f1_score(ytest_target,ytest_pred)))

```

Confusion Matrix of the Test Set



```

-----
[[34345   37]
 [  146    0]]
Precision of the MLP :  0.0
Recall of the MLP    :  0.0
F1 Score of the Model :  0.0

```

Write code to weighted prediction accuracy

That is, sum of the weights of the correct predictions divided by the total weight across all the input elements.

Report the weighted accuracy on the test data.

```
In [20]: #####
#
# Complete the following function which calculates weight prediction accuracy
#
def weight_accuracy(predicted_y, true_y, weight):
    """
    Inputs:
        predicted_y: predicted labels
        true_y: true labels
        weight: weight of each sample
    return:
        sum of the weights of the correct predictions divided by the total weight
    """
    # Convert inputs to numpy arrays if they are tensors
    if isinstance(predicted_y, torch.Tensor):
        predicted_y = predicted_y.cpu().numpy()
    if isinstance(true_y, torch.Tensor):
        true_y = true_y.cpu().numpy()
    if isinstance(weight, torch.Tensor):
        weight = weight.cpu().numpy()

    # Flatten arrays if needed
    predicted_y = np.array(predicted_y).flatten()
    true_y = np.array(true_y).flatten()
    weight = np.array(weight).flatten()

    # Find correct predictions
    correct = (predicted_y == true_y)

    # Sum of weights for correct predictions
    correct_weight_sum = np.sum(weight[correct])

    # Total weight
    total_weight = np.sum(weight)

    # Weighted accuracy
    weighted_acc = correct_weight_sum / total_weight if total_weight > 0 else 0

    return weighted_acc

#####
```

```
In [21]: # calculate weighted accuracy on the test data
import numpy as np

# Get weights from test dataset
test_weights = []
test_labels = []
test_predictions = []
```

```

model.eval()
with torch.no_grad():
    for i, (X, y, w, a) in enumerate(test_loader):
        output = model(X, a)
        y_pred_tag = (output > 0).int()
        test_predictions.append(y_pred_tag.cpu().numpy())
        test_labels.append(y.cpu().numpy())
        test_weights.append(w.cpu().numpy())

# Flatten all lists
test_predictions = np.concatenate([arr.flatten() for arr in test_predictions])
test_labels = np.concatenate([arr.flatten() for arr in test_labels])
test_weights = np.concatenate([arr.flatten() for arr in test_weights])

# Calculate weighted accuracy
weighted_acc = weight_accuracy(test_predictions, test_labels, test_weights)
print(f"Weighted Accuracy on Test Set: {weighted_acc:.4f}")

```

Weighted Accuracy on Test Set: 0.4995

Problem 2 - CNN

- Define a CNN model to solve the above problem.
- The CNN model receives the same inputs. Instead of using MLP, it uses convolution to extract features. *italicized text*
- Different from 2d images, the convolution will be 1d convolution
- Again, use validation set to design your model and select hyperparameters.
- Report weight accuracy on test set
- Comment on your observations.

CNN Details

- For the CNN model, you have to use the 1D convolution module. The in_channels will be 4, one per DNA base. You can decide what number of out_channels and kernel size you want to use.

- Check out torch Conv1d

```
torch.nn.Conv1d(in_channels, out_channels, kernel_size,
stride=1, padding=0)
```

- Note the dimensions of the input required for the 1D convolution -- (N,C,L) a where N is a batch size, C denotes a number of channels, L is a length of signal sequence.
- That is the input sequence has to be 4×101 and not 101×4 as in the data. So you should use torch.swapaxes function to swap the last two axes (not the batch axis) in the forward function.

After the Conv1d, you can apply a relu activation, do dropout, and then try a maxpooling layer (1d). You can try more than one convolution layer too.

Finally, flatten out the last convolution layer and use as input to an MLP.

```
In [22]: #####
# CNN Model
#####
class CNNModel(nn.Module):
    def __init__(self, num_filters=64, kernel_size=5, hidden_size=128):
        super(CNNModel, self).__init__()
        # First conv layer: in_channels=4 (DNA bases), out_channels=num_filt
        self.conv1 = nn.Conv1d(in_channels=4, out_channels=num_filters, kerr
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(0.3)
        self.pool1 = nn.MaxPool1d(kernel_size=2)

        # Second conv layer
        self.conv2 = nn.Conv1d(in_channels=num_filters, out_channels=num_fil
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(0.3)
        self.pool2 = nn.MaxPool1d(kernel_size=2)

        # Calculate the size after convolutions and pooling
        # After pool1: 101 -> 50 (if stride=2)
        # After pool2: 50 -> 25
        # So final size is num_filters*2 * 25
        self.fc1 = nn.Linear(num_filters*2 * 25 + 1, hidden_size) # +1 for
        self.relu3 = nn.ReLU()
        self.dropout3 = nn.Dropout(0.3)
        self.fc2 = nn.Linear(hidden_size, 1)

    def forward(self, X, a):
        # Swap axes: (N, 101, 4) -> (N, 4, 101) for Conv1d
        out = torch.swapaxes(X, 1, 2)

        # First conv block
        out = self.conv1(out)
        out = self.relu1(out)
        out = self.dropout1(out)
        out = self.pool1(out)

        # Second conv block
        out = self.conv2(out)
        out = self.relu2(out)
        out = self.dropout2(out)
        out = self.pool2(out)

        # Flatten
        out = out.view(out.size(0), -1)

        # Concatenate with accessibility
        out = torch.cat((out, a), 1)
```

```

# Fully connected layers
out = self.fc1(out)
out = self.relu3(out)
out = self.dropout3(out)
out = self.fc2(out)

```

```

return out

```

```

#####

```

```

In [23]: # Train CNN Model
cnn_model = CNNModel()
cnn_criterion = nn.BCEWithLogitsLoss(reduction='none')
cnn_optimizer = torch.optim.Adam(cnn_model.parameters(), lr=0.001)

num_epochs_cnn = 10
cnn_model.train()
for epoch in range(num_epochs_cnn):
    for i, (X, y, w, a) in enumerate(train_loader):
        output = cnn_model(X, a)
        loss_per_sample = cnn_criterion(output, y)
        loss = (loss_per_sample * w.squeeze()).mean()

        cnn_optimizer.zero_grad()
        loss.backward()
        cnn_optimizer.step()

        if (i+1) % 100 == 0:
            print(f'CNN Epoch [{epoch+1}/{num_epochs_cnn}], Step [{i+1}/{len(train_loader)}], Loss: {loss.item():.4f}')

    valid_acc = evaluate_model(cnn_model, valid_loader)
    print(f'CNN Epoch [{epoch+1}/{num_epochs_cnn}], Validation Weighted Accuracy: {valid_acc:.4f}')

# Evaluate CNN on test set
cnn_test_acc = evaluate_model(cnn_model, test_loader)
print(f'\nCNN Weighted Accuracy on Test Set: {cnn_test_acc:.4f}')

```

```
CNN Epoch [1/10], Step [100/2763], Loss: 0.0896
CNN Epoch [1/10], Step [200/2763], Loss: 0.0031
CNN Epoch [1/10], Step [300/2763], Loss: 0.1066
CNN Epoch [1/10], Step [400/2763], Loss: 0.0076
CNN Epoch [1/10], Step [500/2763], Loss: 0.0010
CNN Epoch [1/10], Step [600/2763], Loss: 0.0016
CNN Epoch [1/10], Step [700/2763], Loss: 0.0054
CNN Epoch [1/10], Step [800/2763], Loss: 0.0046
CNN Epoch [1/10], Step [900/2763], Loss: 0.0043
CNN Epoch [1/10], Step [1000/2763], Loss: 0.0045
CNN Epoch [1/10], Step [1100/2763], Loss: 0.1074
CNN Epoch [1/10], Step [1200/2763], Loss: 0.1012
CNN Epoch [1/10], Step [1300/2763], Loss: 0.1116
CNN Epoch [1/10], Step [1400/2763], Loss: 0.0033
CNN Epoch [1/10], Step [1500/2763], Loss: 0.0039
CNN Epoch [1/10], Step [1600/2763], Loss: 0.0039
CNN Epoch [1/10], Step [1700/2763], Loss: 0.0051
CNN Epoch [1/10], Step [1800/2763], Loss: 0.1388
CNN Epoch [1/10], Step [1900/2763], Loss: 0.0072
CNN Epoch [1/10], Step [2000/2763], Loss: 0.0080
CNN Epoch [1/10], Step [2100/2763], Loss: 0.0022
CNN Epoch [1/10], Step [2200/2763], Loss: 0.1095
CNN Epoch [1/10], Step [2300/2763], Loss: 0.1130
CNN Epoch [1/10], Step [2400/2763], Loss: 0.0078
CNN Epoch [1/10], Step [2500/2763], Loss: 0.1080
CNN Epoch [1/10], Step [2600/2763], Loss: 0.0805
CNN Epoch [1/10], Step [2700/2763], Loss: 0.0034
CNN Epoch [1/10], Validation Weighted Accuracy: 0.5000
CNN Epoch [2/10], Step [100/2763], Loss: 0.0022
CNN Epoch [2/10], Step [200/2763], Loss: 0.0046
CNN Epoch [2/10], Step [300/2763], Loss: 0.0020
CNN Epoch [2/10], Step [400/2763], Loss: 0.0038
CNN Epoch [2/10], Step [500/2763], Loss: 0.0094
CNN Epoch [2/10], Step [600/2763], Loss: 0.0024
CNN Epoch [2/10], Step [700/2763], Loss: 0.0045
CNN Epoch [2/10], Step [800/2763], Loss: 0.0026
CNN Epoch [2/10], Step [900/2763], Loss: 0.0951
CNN Epoch [2/10], Step [1000/2763], Loss: 0.0049
CNN Epoch [2/10], Step [1100/2763], Loss: 0.0956
CNN Epoch [2/10], Step [1200/2763], Loss: 0.0023
CNN Epoch [2/10], Step [1300/2763], Loss: 0.0037
CNN Epoch [2/10], Step [1400/2763], Loss: 0.0028
CNN Epoch [2/10], Step [1500/2763], Loss: 0.1000
CNN Epoch [2/10], Step [1600/2763], Loss: 0.0029
CNN Epoch [2/10], Step [1700/2763], Loss: 0.0963
CNN Epoch [2/10], Step [1800/2763], Loss: 0.0073
CNN Epoch [2/10], Step [1900/2763], Loss: 0.0061
CNN Epoch [2/10], Step [2000/2763], Loss: 0.0025
CNN Epoch [2/10], Step [2100/2763], Loss: 0.0978
CNN Epoch [2/10], Step [2200/2763], Loss: 0.0027
CNN Epoch [2/10], Step [2300/2763], Loss: 0.0864
CNN Epoch [2/10], Step [2400/2763], Loss: 0.0033
CNN Epoch [2/10], Step [2500/2763], Loss: 0.0044
CNN Epoch [2/10], Step [2600/2763], Loss: 0.0072
CNN Epoch [2/10], Step [2700/2763], Loss: 0.0032
CNN Epoch [2/10], Validation Weighted Accuracy: 0.5000
```

```
CNN Epoch [3/10], Step [100/2763], Loss: 0.0030
CNN Epoch [3/10], Step [200/2763], Loss: 0.0064
CNN Epoch [3/10], Step [300/2763], Loss: 0.0036
CNN Epoch [3/10], Step [400/2763], Loss: 0.0065
CNN Epoch [3/10], Step [500/2763], Loss: 0.0085
CNN Epoch [3/10], Step [600/2763], Loss: 0.0051
CNN Epoch [3/10], Step [700/2763], Loss: 0.0034
CNN Epoch [3/10], Step [800/2763], Loss: 0.0035
CNN Epoch [3/10], Step [900/2763], Loss: 0.0032
CNN Epoch [3/10], Step [1000/2763], Loss: 0.0906
CNN Epoch [3/10], Step [1100/2763], Loss: 0.0826
CNN Epoch [3/10], Step [1200/2763], Loss: 0.0972
CNN Epoch [3/10], Step [1300/2763], Loss: 0.0021
CNN Epoch [3/10], Step [1400/2763], Loss: 0.0061
CNN Epoch [3/10], Step [1500/2763], Loss: 0.0063
CNN Epoch [3/10], Step [1600/2763], Loss: 0.0031
CNN Epoch [3/10], Step [1700/2763], Loss: 0.0037
CNN Epoch [3/10], Step [1800/2763], Loss: 0.0045
CNN Epoch [3/10], Step [1900/2763], Loss: 0.0061
CNN Epoch [3/10], Step [2000/2763], Loss: 0.0055
CNN Epoch [3/10], Step [2100/2763], Loss: 0.0028
CNN Epoch [3/10], Step [2200/2763], Loss: 0.0069
CNN Epoch [3/10], Step [2300/2763], Loss: 0.0935
CNN Epoch [3/10], Step [2400/2763], Loss: 0.0814
CNN Epoch [3/10], Step [2500/2763], Loss: 0.0062
CNN Epoch [3/10], Step [2600/2763], Loss: 0.0018
CNN Epoch [3/10], Step [2700/2763], Loss: 0.0927
CNN Epoch [3/10], Validation Weighted Accuracy: 0.5000
CNN Epoch [4/10], Step [100/2763], Loss: 0.0946
CNN Epoch [4/10], Step [200/2763], Loss: 0.0069
CNN Epoch [4/10], Step [300/2763], Loss: 0.0977
CNN Epoch [4/10], Step [400/2763], Loss: 0.0044
CNN Epoch [4/10], Step [500/2763], Loss: 0.0019
CNN Epoch [4/10], Step [600/2763], Loss: 0.0798
CNN Epoch [4/10], Step [700/2763], Loss: 0.0074
CNN Epoch [4/10], Step [800/2763], Loss: 0.0089
CNN Epoch [4/10], Step [900/2763], Loss: 0.0028
CNN Epoch [4/10], Step [1000/2763], Loss: 0.0050
CNN Epoch [4/10], Step [1100/2763], Loss: 0.0036
CNN Epoch [4/10], Step [1200/2763], Loss: 0.0129
CNN Epoch [4/10], Step [1300/2763], Loss: 0.0057
CNN Epoch [4/10], Step [1400/2763], Loss: 0.0030
CNN Epoch [4/10], Step [1500/2763], Loss: 0.0043
CNN Epoch [4/10], Step [1600/2763], Loss: 0.0056
CNN Epoch [4/10], Step [1700/2763], Loss: 0.0093
CNN Epoch [4/10], Step [1800/2763], Loss: 0.0082
CNN Epoch [4/10], Step [1900/2763], Loss: 0.0770
CNN Epoch [4/10], Step [2000/2763], Loss: 0.1210
CNN Epoch [4/10], Step [2100/2763], Loss: 0.0087
CNN Epoch [4/10], Step [2200/2763], Loss: 0.0072
CNN Epoch [4/10], Step [2300/2763], Loss: 0.0039
CNN Epoch [4/10], Step [2400/2763], Loss: 0.0031
CNN Epoch [4/10], Step [2500/2763], Loss: 0.0666
CNN Epoch [4/10], Step [2600/2763], Loss: 0.0033
CNN Epoch [4/10], Step [2700/2763], Loss: 0.0074
CNN Epoch [4/10], Validation Weighted Accuracy: 0.5000
```

CNN Epoch [5/10], Step [100/2763], Loss: 0.0661
CNN Epoch [5/10], Step [200/2763], Loss: 0.0019
CNN Epoch [5/10], Step [300/2763], Loss: 0.0025
CNN Epoch [5/10], Step [400/2763], Loss: 0.0029
CNN Epoch [5/10], Step [500/2763], Loss: 0.0014
CNN Epoch [5/10], Step [600/2763], Loss: 0.0051
CNN Epoch [5/10], Step [700/2763], Loss: 0.0549
CNN Epoch [5/10], Step [800/2763], Loss: 0.0037
CNN Epoch [5/10], Step [900/2763], Loss: 0.0057
CNN Epoch [5/10], Step [1000/2763], Loss: 0.1021
CNN Epoch [5/10], Step [1100/2763], Loss: 0.0540
CNN Epoch [5/10], Step [1200/2763], Loss: 0.0566
CNN Epoch [5/10], Step [1300/2763], Loss: 0.0064
CNN Epoch [5/10], Step [1400/2763], Loss: 0.0058
CNN Epoch [5/10], Step [1500/2763], Loss: 0.0024
CNN Epoch [5/10], Step [1600/2763], Loss: 0.0046
CNN Epoch [5/10], Step [1700/2763], Loss: 0.0029
CNN Epoch [5/10], Step [1800/2763], Loss: 0.0029
CNN Epoch [5/10], Step [1900/2763], Loss: 0.0012
CNN Epoch [5/10], Step [2000/2763], Loss: 0.0026
CNN Epoch [5/10], Step [2100/2763], Loss: 0.0020
CNN Epoch [5/10], Step [2200/2763], Loss: 0.0028
CNN Epoch [5/10], Step [2300/2763], Loss: 0.0033
CNN Epoch [5/10], Step [2400/2763], Loss: 0.0043
CNN Epoch [5/10], Step [2500/2763], Loss: 0.5736
CNN Epoch [5/10], Step [2600/2763], Loss: 0.0713
CNN Epoch [5/10], Step [2700/2763], Loss: 0.1985
CNN Epoch [5/10], Validation Weighted Accuracy: 0.5000
CNN Epoch [6/10], Step [100/2763], Loss: 0.0425
CNN Epoch [6/10], Step [200/2763], Loss: 0.1279
CNN Epoch [6/10], Step [300/2763], Loss: 0.0058
CNN Epoch [6/10], Step [400/2763], Loss: 0.0035
CNN Epoch [6/10], Step [500/2763], Loss: 0.0043
CNN Epoch [6/10], Step [600/2763], Loss: 0.0022
CNN Epoch [6/10], Step [700/2763], Loss: 0.0501
CNN Epoch [6/10], Step [800/2763], Loss: 0.0015
CNN Epoch [6/10], Step [900/2763], Loss: 0.0184
CNN Epoch [6/10], Step [1000/2763], Loss: 0.0666
CNN Epoch [6/10], Step [1100/2763], Loss: 0.0043
CNN Epoch [6/10], Step [1200/2763], Loss: 0.0294
CNN Epoch [6/10], Step [1300/2763], Loss: 0.0962
CNN Epoch [6/10], Step [1400/2763], Loss: 0.0067
CNN Epoch [6/10], Step [1500/2763], Loss: 0.0333
CNN Epoch [6/10], Step [1600/2763], Loss: 0.0402
CNN Epoch [6/10], Step [1700/2763], Loss: 0.0051
CNN Epoch [6/10], Step [1800/2763], Loss: 0.0215
CNN Epoch [6/10], Step [1900/2763], Loss: 0.0024
CNN Epoch [6/10], Step [2000/2763], Loss: 0.0028
CNN Epoch [6/10], Step [2100/2763], Loss: 0.0015
CNN Epoch [6/10], Step [2200/2763], Loss: 0.0067
CNN Epoch [6/10], Step [2300/2763], Loss: 0.0023
CNN Epoch [6/10], Step [2400/2763], Loss: 0.0309
CNN Epoch [6/10], Step [2500/2763], Loss: 0.0087
CNN Epoch [6/10], Step [2600/2763], Loss: 0.0036
CNN Epoch [6/10], Step [2700/2763], Loss: 0.0026
CNN Epoch [6/10], Validation Weighted Accuracy: 0.5000

CNN Epoch [7/10], Step [100/2763], Loss: 0.0011
CNN Epoch [7/10], Step [200/2763], Loss: 0.0214
CNN Epoch [7/10], Step [300/2763], Loss: 0.1176
CNN Epoch [7/10], Step [400/2763], Loss: 0.0021
CNN Epoch [7/10], Step [500/2763], Loss: 0.0002
CNN Epoch [7/10], Step [600/2763], Loss: 0.0866
CNN Epoch [7/10], Step [700/2763], Loss: 0.0015
CNN Epoch [7/10], Step [800/2763], Loss: 0.0057
CNN Epoch [7/10], Step [900/2763], Loss: 0.0322
CNN Epoch [7/10], Step [1000/2763], Loss: 0.0007
CNN Epoch [7/10], Step [1100/2763], Loss: 0.0039
CNN Epoch [7/10], Step [1200/2763], Loss: 0.0003
CNN Epoch [7/10], Step [1300/2763], Loss: 0.0610
CNN Epoch [7/10], Step [1400/2763], Loss: 0.0011
CNN Epoch [7/10], Step [1500/2763], Loss: 0.0051
CNN Epoch [7/10], Step [1600/2763], Loss: 0.0255
CNN Epoch [7/10], Step [1700/2763], Loss: 0.0023
CNN Epoch [7/10], Step [1800/2763], Loss: 0.1437
CNN Epoch [7/10], Step [1900/2763], Loss: 0.0040
CNN Epoch [7/10], Step [2000/2763], Loss: 0.0201
CNN Epoch [7/10], Step [2100/2763], Loss: 0.0009
CNN Epoch [7/10], Step [2200/2763], Loss: 0.0014
CNN Epoch [7/10], Step [2300/2763], Loss: 0.0564
CNN Epoch [7/10], Step [2400/2763], Loss: 0.0058
CNN Epoch [7/10], Step [2500/2763], Loss: 0.0013
CNN Epoch [7/10], Step [2600/2763], Loss: 0.0247
CNN Epoch [7/10], Step [2700/2763], Loss: 0.0028
CNN Epoch [7/10], Validation Weighted Accuracy: 0.5064
CNN Epoch [8/10], Step [100/2763], Loss: 0.0009
CNN Epoch [8/10], Step [200/2763], Loss: 0.0005
CNN Epoch [8/10], Step [300/2763], Loss: 0.0016
CNN Epoch [8/10], Step [400/2763], Loss: 0.0002
CNN Epoch [8/10], Step [500/2763], Loss: 0.0104
CNN Epoch [8/10], Step [600/2763], Loss: 0.0002
CNN Epoch [8/10], Step [700/2763], Loss: 0.0046
CNN Epoch [8/10], Step [800/2763], Loss: 0.0074
CNN Epoch [8/10], Step [900/2763], Loss: 0.0011
CNN Epoch [8/10], Step [1000/2763], Loss: 0.0656
CNN Epoch [8/10], Step [1100/2763], Loss: 0.0121
CNN Epoch [8/10], Step [1200/2763], Loss: 0.0002
CNN Epoch [8/10], Step [1300/2763], Loss: 0.1002
CNN Epoch [8/10], Step [1400/2763], Loss: 0.0098
CNN Epoch [8/10], Step [1500/2763], Loss: 0.0034
CNN Epoch [8/10], Step [1600/2763], Loss: 0.0096
CNN Epoch [8/10], Step [1700/2763], Loss: 0.0662
CNN Epoch [8/10], Step [1800/2763], Loss: 0.0026
CNN Epoch [8/10], Step [1900/2763], Loss: 0.0577
CNN Epoch [8/10], Step [2000/2763], Loss: 0.0028
CNN Epoch [8/10], Step [2100/2763], Loss: 0.0032
CNN Epoch [8/10], Step [2200/2763], Loss: 0.0136
CNN Epoch [8/10], Step [2300/2763], Loss: 0.0014
CNN Epoch [8/10], Step [2400/2763], Loss: 0.0233
CNN Epoch [8/10], Step [2500/2763], Loss: 0.0011
CNN Epoch [8/10], Step [2600/2763], Loss: 0.0075
CNN Epoch [8/10], Step [2700/2763], Loss: 0.0263
CNN Epoch [8/10], Validation Weighted Accuracy: 0.5031

```
CNN Epoch [9/10], Step [100/2763], Loss: 0.0004
CNN Epoch [9/10], Step [200/2763], Loss: 0.0097
CNN Epoch [9/10], Step [300/2763], Loss: 0.0085
CNN Epoch [9/10], Step [400/2763], Loss: 0.0003
CNN Epoch [9/10], Step [500/2763], Loss: 0.0000
CNN Epoch [9/10], Step [600/2763], Loss: 0.0207
CNN Epoch [9/10], Step [700/2763], Loss: 0.0009
CNN Epoch [9/10], Step [800/2763], Loss: 0.0267
CNN Epoch [9/10], Step [900/2763], Loss: 0.0026
CNN Epoch [9/10], Step [1000/2763], Loss: 0.0004
CNN Epoch [9/10], Step [1100/2763], Loss: 0.0273
CNN Epoch [9/10], Step [1200/2763], Loss: 0.0061
CNN Epoch [9/10], Step [1300/2763], Loss: 0.0007
CNN Epoch [9/10], Step [1400/2763], Loss: 0.0404
CNN Epoch [9/10], Step [1500/2763], Loss: 0.0031
CNN Epoch [9/10], Step [1600/2763], Loss: 0.1074
CNN Epoch [9/10], Step [1700/2763], Loss: 0.0086
CNN Epoch [9/10], Step [1800/2763], Loss: 0.0002
CNN Epoch [9/10], Step [1900/2763], Loss: 0.0397
CNN Epoch [9/10], Step [2000/2763], Loss: 0.0046
CNN Epoch [9/10], Step [2100/2763], Loss: 0.0125
CNN Epoch [9/10], Step [2200/2763], Loss: 0.0002
CNN Epoch [9/10], Step [2300/2763], Loss: 0.0200
CNN Epoch [9/10], Step [2400/2763], Loss: 0.0051
CNN Epoch [9/10], Step [2500/2763], Loss: 0.0028
CNN Epoch [9/10], Step [2600/2763], Loss: 0.0104
CNN Epoch [9/10], Step [2700/2763], Loss: 0.1325
CNN Epoch [9/10], Validation Weighted Accuracy: 0.5026
CNN Epoch [10/10], Step [100/2763], Loss: 0.0022
CNN Epoch [10/10], Step [200/2763], Loss: 0.0000
CNN Epoch [10/10], Step [300/2763], Loss: 0.0001
CNN Epoch [10/10], Step [400/2763], Loss: 0.0189
CNN Epoch [10/10], Step [500/2763], Loss: 0.0312
CNN Epoch [10/10], Step [600/2763], Loss: 0.0011
CNN Epoch [10/10], Step [700/2763], Loss: 0.0321
CNN Epoch [10/10], Step [800/2763], Loss: 0.0002
CNN Epoch [10/10], Step [900/2763], Loss: 0.0005
CNN Epoch [10/10], Step [1000/2763], Loss: 0.0001
CNN Epoch [10/10], Step [1100/2763], Loss: 0.0001
CNN Epoch [10/10], Step [1200/2763], Loss: 0.0142
CNN Epoch [10/10], Step [1300/2763], Loss: 0.0003
CNN Epoch [10/10], Step [1400/2763], Loss: 0.0002
CNN Epoch [10/10], Step [1500/2763], Loss: 0.0013
CNN Epoch [10/10], Step [1600/2763], Loss: 0.0025
CNN Epoch [10/10], Step [1700/2763], Loss: 0.0022
CNN Epoch [10/10], Step [1800/2763], Loss: 0.0081
CNN Epoch [10/10], Step [1900/2763], Loss: 0.0692
CNN Epoch [10/10], Step [2000/2763], Loss: 0.0002
CNN Epoch [10/10], Step [2100/2763], Loss: 0.0071
CNN Epoch [10/10], Step [2200/2763], Loss: 0.0006
CNN Epoch [10/10], Step [2300/2763], Loss: 0.0006
CNN Epoch [10/10], Step [2400/2763], Loss: 0.0378
CNN Epoch [10/10], Step [2500/2763], Loss: 0.0360
CNN Epoch [10/10], Step [2600/2763], Loss: 0.0010
CNN Epoch [10/10], Step [2700/2763], Loss: 0.0018
CNN Epoch [10/10], Validation Weighted Accuracy: 0.5024
```

CNN Weighted Accuracy on Test Set: 0.5229

Problem 3 - LSTM

- For the LSTM model, use the encoder followed by a two layer MLP approach. That is, pass the input sequence (batch) through the LSTM and use the last hidden layer as the representation or embedding vector for the sequence. You can choose the dimensionality of the hidden layer. Next, use this vector as input to a two fully connected MLP layers -- the first connects the input vector to the hidden layer (again you can choose the size of the hidden layer), and the second connects the hidden to the output neuron. Use dropout and relu as appropriate.
- Keep in mind that for the input to the LSTM module in pytorch use batch_first=True. This means that the batch dimension comes first, so the input is (N×101×4), which is how the input data is structured. Make note of the output of the LSTM layer so that you store the last hidden layer as the representation, to be used as input to the MLP layers.
- Also, before feeding the output of the hidden layer to the output layer, you must concatenate the accessibility value. So if you are using hidden dimension of 128, then after concatenating the accessibility value, it will become a 129d vector, which should be fed to the final output layer of size 1, since we have a binary class/label.
- You should use binary_cross_entropy_with_logits with weight set to the weights per input element.
- You need to train the model on the training data, and use the validation data to select how many epochs you want to use and to choose the hidden dimension. Use the weighted prediction accuracy as the evaluation metric. That is, sum of the weights of the correct predictions divided by the total weight across all the input elements. Finally, report the weighted accuracy on the test data.

```
In [24]: #####
# LSTM Model
#####
class LSTMModel(nn.Module):
    def __init__(self, lstm_hidden_size=128, mlp_hidden_size=64):
        super(LSTMModel, self).__init__()
        # LSTM layer: input_size=4 (DNA bases), hidden_size=lstm_hidden_size
        # batch_first=True means input is (batch, seq_len, input_size)
        self.lstm = nn.LSTM(input_size=4, hidden_size=lstm_hidden_size,
                             batch_first=True, num_layers=1)

        # Two layer MLP
        # Input: lstm_hidden_size + 1 (accessibility)
        self.fc1 = nn.Linear(lstm_hidden_size + 1, mlp_hidden_size)
```

```

self.relu = nn.ReLU()
self.dropout = nn.Dropout(0.3)
self.fc2 = nn.Linear(mlp_hidden_size, 1)

def forward(self, X, a):
    # X shape: (batch, 101, 4) - already in batch_first format
    # Pass through LSTM
    lstm_out, (hidden, cell) = self.lstm(X)
    # Use the last hidden state from the last layer
    # hidden shape: (num_layers, batch, hidden_size)
    # We want the last layer's hidden state
    lstm_hidden = hidden[-1] # Shape: (batch, hidden_size)

    # Concatenate with accessibility
    # Ensure a is 2D: a comes as (batch, 1), but we need to ensure it st
    # If a was squeezed somewhere, reshape it back to (batch, 1)
    if a.dim() == 1:
        a = a.unsqueeze(1) # (batch,) -> (batch, 1)
    # a should already be (batch, 1) from DataLoader, so we can use it c

    out = torch.cat((lstm_hidden, a), 1) # Shape: (batch, hidden_size+1

    # Pass through MLP
    out = self.fc1(out)
    out = self.relu(out)
    out = self.dropout(out)
    out = self.fc2(out)

    return out
#####

# Train LSTM Model
lstm_model = LSTMModel()
lstm_criterion = nn.BCEWithLogitsLoss(reduction='none')
lstm_optimizer = torch.optim.Adam(lstm_model.parameters(), lr=0.001)

num_epochs_lstm = 10
lstm_model.train()
for epoch in range(num_epochs_lstm):
    for i, (X, y, w, a) in enumerate(train_loader):
        output = lstm_model(X, a)
        loss_per_sample = lstm_criterion(output, y)
        loss = (loss_per_sample * w.squeeze()).mean()

        lstm_optimizer.zero_grad()
        loss.backward()
        lstm_optimizer.step()

        if (i+1) % 100 == 0:
            print(f'LSTM Epoch [{epoch+1}/{num_epochs_lstm}], Step [{i+1}/{1

    valid_acc = evaluate_model(lstm_model, valid_loader)
    print(f'LSTM Epoch [{epoch+1}/{num_epochs_lstm}], Validation Weighted Ac

# Evaluate LSTM on test set

```

```
lstm_test_acc = evaluate_model(lstm_model, test_loader)
print(f'\nLSTM Weighted Accuracy on Test Set: {lstm_test_acc:.4f}')
```

```
LSTM Epoch [1/10], Step [100/2763], Loss: 0.0990
LSTM Epoch [1/10], Step [200/2763], Loss: 0.1218
LSTM Epoch [1/10], Step [300/2763], Loss: 0.0169
LSTM Epoch [1/10], Step [400/2763], Loss: 0.0023
LSTM Epoch [1/10], Step [500/2763], Loss: 0.3248
LSTM Epoch [1/10], Step [600/2763], Loss: 0.0819
LSTM Epoch [1/10], Step [700/2763], Loss: 0.0038
LSTM Epoch [1/10], Step [800/2763], Loss: 0.0042
LSTM Epoch [1/10], Step [900/2763], Loss: 0.0911
LSTM Epoch [1/10], Step [1000/2763], Loss: 0.0994
LSTM Epoch [1/10], Step [1100/2763], Loss: 0.0861
LSTM Epoch [1/10], Step [1200/2763], Loss: 0.0030
LSTM Epoch [1/10], Step [1300/2763], Loss: 0.1163
LSTM Epoch [1/10], Step [1400/2763], Loss: 0.0024
LSTM Epoch [1/10], Step [1500/2763], Loss: 0.0038
LSTM Epoch [1/10], Step [1600/2763], Loss: 0.0034
LSTM Epoch [1/10], Step [1700/2763], Loss: 0.0080
LSTM Epoch [1/10], Step [1800/2763], Loss: 0.0057
LSTM Epoch [1/10], Step [1900/2763], Loss: 0.0032
LSTM Epoch [1/10], Step [2000/2763], Loss: 0.0043
LSTM Epoch [1/10], Step [2100/2763], Loss: 0.0892
LSTM Epoch [1/10], Step [2200/2763], Loss: 0.0980
LSTM Epoch [1/10], Step [2300/2763], Loss: 0.0043
LSTM Epoch [1/10], Step [2400/2763], Loss: 0.0907
LSTM Epoch [1/10], Step [2500/2763], Loss: 0.2869
LSTM Epoch [1/10], Step [2600/2763], Loss: 0.0049
LSTM Epoch [1/10], Step [2700/2763], Loss: 0.0031
LSTM Epoch [1/10], Validation Weighted Accuracy: 0.5000
LSTM Epoch [2/10], Step [100/2763], Loss: 0.0055
LSTM Epoch [2/10], Step [200/2763], Loss: 0.0063
LSTM Epoch [2/10], Step [300/2763], Loss: 0.0987
LSTM Epoch [2/10], Step [400/2763], Loss: 0.0941
LSTM Epoch [2/10], Step [500/2763], Loss: 0.0043
LSTM Epoch [2/10], Step [600/2763], Loss: 0.0043
LSTM Epoch [2/10], Step [700/2763], Loss: 0.0047
LSTM Epoch [2/10], Step [800/2763], Loss: 0.0055
LSTM Epoch [2/10], Step [900/2763], Loss: 0.0943
LSTM Epoch [2/10], Step [1000/2763], Loss: 0.0076
LSTM Epoch [2/10], Step [1100/2763], Loss: 0.0039
LSTM Epoch [2/10], Step [1200/2763], Loss: 0.0958
LSTM Epoch [2/10], Step [1300/2763], Loss: 0.0049
LSTM Epoch [2/10], Step [1400/2763], Loss: 0.0036
LSTM Epoch [2/10], Step [1500/2763], Loss: 0.0122
LSTM Epoch [2/10], Step [1600/2763], Loss: 0.0036
LSTM Epoch [2/10], Step [1700/2763], Loss: 0.0949
LSTM Epoch [2/10], Step [1800/2763], Loss: 0.0034
LSTM Epoch [2/10], Step [1900/2763], Loss: 0.0035
LSTM Epoch [2/10], Step [2000/2763], Loss: 0.0072
LSTM Epoch [2/10], Step [2100/2763], Loss: 0.0056
LSTM Epoch [2/10], Step [2200/2763], Loss: 0.0098
LSTM Epoch [2/10], Step [2300/2763], Loss: 0.0945
LSTM Epoch [2/10], Step [2400/2763], Loss: 0.0042
LSTM Epoch [2/10], Step [2500/2763], Loss: 0.0053
LSTM Epoch [2/10], Step [2600/2763], Loss: 0.0039
LSTM Epoch [2/10], Step [2700/2763], Loss: 0.0994
LSTM Epoch [2/10], Validation Weighted Accuracy: 0.5000
```

LSTM Epoch [3/10], Step [100/2763], Loss: 0.0941
LSTM Epoch [3/10], Step [200/2763], Loss: 0.0038
LSTM Epoch [3/10], Step [300/2763], Loss: 0.0939
LSTM Epoch [3/10], Step [400/2763], Loss: 0.0042
LSTM Epoch [3/10], Step [500/2763], Loss: 0.2919
LSTM Epoch [3/10], Step [600/2763], Loss: 0.0035
LSTM Epoch [3/10], Step [700/2763], Loss: 0.2842
LSTM Epoch [3/10], Step [800/2763], Loss: 0.0063
LSTM Epoch [3/10], Step [900/2763], Loss: 0.0065
LSTM Epoch [3/10], Step [1000/2763], Loss: 0.0040
LSTM Epoch [3/10], Step [1100/2763], Loss: 0.0043
LSTM Epoch [3/10], Step [1200/2763], Loss: 0.0973
LSTM Epoch [3/10], Step [1300/2763], Loss: 0.0037
LSTM Epoch [3/10], Step [1400/2763], Loss: 0.0042
LSTM Epoch [3/10], Step [1500/2763], Loss: 0.0054
LSTM Epoch [3/10], Step [1600/2763], Loss: 0.0036
LSTM Epoch [3/10], Step [1700/2763], Loss: 0.0085
LSTM Epoch [3/10], Step [1800/2763], Loss: 0.0954
LSTM Epoch [3/10], Step [1900/2763], Loss: 0.0045
LSTM Epoch [3/10], Step [2000/2763], Loss: 0.0030
LSTM Epoch [3/10], Step [2100/2763], Loss: 0.0045
LSTM Epoch [3/10], Step [2200/2763], Loss: 0.0021
LSTM Epoch [3/10], Step [2300/2763], Loss: 0.0028
LSTM Epoch [3/10], Step [2400/2763], Loss: 0.0036
LSTM Epoch [3/10], Step [2500/2763], Loss: 0.0941
LSTM Epoch [3/10], Step [2600/2763], Loss: 0.0058
LSTM Epoch [3/10], Step [2700/2763], Loss: 0.0940
LSTM Epoch [3/10], Validation Weighted Accuracy: 0.5000
LSTM Epoch [4/10], Step [100/2763], Loss: 0.0041
LSTM Epoch [4/10], Step [200/2763], Loss: 0.0946
LSTM Epoch [4/10], Step [300/2763], Loss: 0.0050
LSTM Epoch [4/10], Step [400/2763], Loss: 0.0090
LSTM Epoch [4/10], Step [500/2763], Loss: 0.0948
LSTM Epoch [4/10], Step [600/2763], Loss: 0.0949
LSTM Epoch [4/10], Step [700/2763], Loss: 0.0934
LSTM Epoch [4/10], Step [800/2763], Loss: 0.0943
LSTM Epoch [4/10], Step [900/2763], Loss: 0.0036
LSTM Epoch [4/10], Step [1000/2763], Loss: 0.0032
LSTM Epoch [4/10], Step [1100/2763], Loss: 0.0972
LSTM Epoch [4/10], Step [1200/2763], Loss: 0.0026
LSTM Epoch [4/10], Step [1300/2763], Loss: 0.0064
LSTM Epoch [4/10], Step [1400/2763], Loss: 0.0050
LSTM Epoch [4/10], Step [1500/2763], Loss: 0.0037
LSTM Epoch [4/10], Step [1600/2763], Loss: 0.2876
LSTM Epoch [4/10], Step [1700/2763], Loss: 0.0945
LSTM Epoch [4/10], Step [1800/2763], Loss: 0.0046
LSTM Epoch [4/10], Step [1900/2763], Loss: 0.0937
LSTM Epoch [4/10], Step [2000/2763], Loss: 0.0068
LSTM Epoch [4/10], Step [2100/2763], Loss: 0.0045
LSTM Epoch [4/10], Step [2200/2763], Loss: 0.0054
LSTM Epoch [4/10], Step [2300/2763], Loss: 0.0058
LSTM Epoch [4/10], Step [2400/2763], Loss: 0.0947
LSTM Epoch [4/10], Step [2500/2763], Loss: 0.0025
LSTM Epoch [4/10], Step [2600/2763], Loss: 0.0058
LSTM Epoch [4/10], Step [2700/2763], Loss: 0.0061
LSTM Epoch [4/10], Validation Weighted Accuracy: 0.5000

```
LSTM Epoch [5/10], Step [100/2763], Loss: 0.0963
LSTM Epoch [5/10], Step [200/2763], Loss: 0.0053
LSTM Epoch [5/10], Step [300/2763], Loss: 0.0047
LSTM Epoch [5/10], Step [400/2763], Loss: 0.0048
LSTM Epoch [5/10], Step [500/2763], Loss: 0.0938
LSTM Epoch [5/10], Step [600/2763], Loss: 0.5950
LSTM Epoch [5/10], Step [700/2763], Loss: 0.0947
LSTM Epoch [5/10], Step [800/2763], Loss: 0.0949
LSTM Epoch [5/10], Step [900/2763], Loss: 0.0938
LSTM Epoch [5/10], Step [1000/2763], Loss: 0.0036
LSTM Epoch [5/10], Step [1100/2763], Loss: 0.0939
LSTM Epoch [5/10], Step [1200/2763], Loss: 0.0059
LSTM Epoch [5/10], Step [1300/2763], Loss: 0.0045
LSTM Epoch [5/10], Step [1400/2763], Loss: 0.0914
LSTM Epoch [5/10], Step [1500/2763], Loss: 0.0937
LSTM Epoch [5/10], Step [1600/2763], Loss: 0.0042
LSTM Epoch [5/10], Step [1700/2763], Loss: 0.0025
LSTM Epoch [5/10], Step [1800/2763], Loss: 0.0046
LSTM Epoch [5/10], Step [1900/2763], Loss: 0.0037
LSTM Epoch [5/10], Step [2000/2763], Loss: 0.0928
LSTM Epoch [5/10], Step [2100/2763], Loss: 0.0921
LSTM Epoch [5/10], Step [2200/2763], Loss: 0.0947
LSTM Epoch [5/10], Step [2300/2763], Loss: 0.0176
LSTM Epoch [5/10], Step [2400/2763], Loss: 0.0937
LSTM Epoch [5/10], Step [2500/2763], Loss: 0.0941
LSTM Epoch [5/10], Step [2600/2763], Loss: 0.0029
LSTM Epoch [5/10], Step [2700/2763], Loss: 0.0062
LSTM Epoch [5/10], Validation Weighted Accuracy: 0.5000
LSTM Epoch [6/10], Step [100/2763], Loss: 0.0946
LSTM Epoch [6/10], Step [200/2763], Loss: 0.0040
LSTM Epoch [6/10], Step [300/2763], Loss: 0.0044
LSTM Epoch [6/10], Step [400/2763], Loss: 0.0039
LSTM Epoch [6/10], Step [500/2763], Loss: 0.2932
LSTM Epoch [6/10], Step [600/2763], Loss: 0.0062
LSTM Epoch [6/10], Step [700/2763], Loss: 0.0044
LSTM Epoch [6/10], Step [800/2763], Loss: 0.0035
LSTM Epoch [6/10], Step [900/2763], Loss: 0.0946
LSTM Epoch [6/10], Step [1000/2763], Loss: 0.0034
LSTM Epoch [6/10], Step [1100/2763], Loss: 0.0040
LSTM Epoch [6/10], Step [1200/2763], Loss: 0.0040
LSTM Epoch [6/10], Step [1300/2763], Loss: 0.0049
LSTM Epoch [6/10], Step [1400/2763], Loss: 0.0944
LSTM Epoch [6/10], Step [1500/2763], Loss: 0.0055
LSTM Epoch [6/10], Step [1600/2763], Loss: 0.0930
LSTM Epoch [6/10], Step [1700/2763], Loss: 0.0921
LSTM Epoch [6/10], Step [1800/2763], Loss: 0.0033
LSTM Epoch [6/10], Step [1900/2763], Loss: 0.0941
LSTM Epoch [6/10], Step [2000/2763], Loss: 0.0940
LSTM Epoch [6/10], Step [2100/2763], Loss: 0.0054
LSTM Epoch [6/10], Step [2200/2763], Loss: 0.0046
LSTM Epoch [6/10], Step [2300/2763], Loss: 0.0057
LSTM Epoch [6/10], Step [2400/2763], Loss: 0.0033
LSTM Epoch [6/10], Step [2500/2763], Loss: 0.0038
LSTM Epoch [6/10], Step [2600/2763], Loss: 0.0039
LSTM Epoch [6/10], Step [2700/2763], Loss: 0.0033
LSTM Epoch [6/10], Validation Weighted Accuracy: 0.5000
```

LSTM Epoch [7/10], Step [100/2763], Loss: 0.2891
LSTM Epoch [7/10], Step [200/2763], Loss: 0.0924
LSTM Epoch [7/10], Step [300/2763], Loss: 0.0038
LSTM Epoch [7/10], Step [400/2763], Loss: 0.0047
LSTM Epoch [7/10], Step [500/2763], Loss: 0.0029
LSTM Epoch [7/10], Step [600/2763], Loss: 0.0047
LSTM Epoch [7/10], Step [700/2763], Loss: 0.0034
LSTM Epoch [7/10], Step [800/2763], Loss: 0.0928
LSTM Epoch [7/10], Step [900/2763], Loss: 0.0035
LSTM Epoch [7/10], Step [1000/2763], Loss: 0.0044
LSTM Epoch [7/10], Step [1100/2763], Loss: 0.0031
LSTM Epoch [7/10], Step [1200/2763], Loss: 0.0057
LSTM Epoch [7/10], Step [1300/2763], Loss: 0.0048
LSTM Epoch [7/10], Step [1400/2763], Loss: 0.0040
LSTM Epoch [7/10], Step [1500/2763], Loss: 0.0934
LSTM Epoch [7/10], Step [1600/2763], Loss: 0.0938
LSTM Epoch [7/10], Step [1700/2763], Loss: 0.0053
LSTM Epoch [7/10], Step [1800/2763], Loss: 0.0043
LSTM Epoch [7/10], Step [1900/2763], Loss: 0.0935
LSTM Epoch [7/10], Step [2000/2763], Loss: 0.0059
LSTM Epoch [7/10], Step [2100/2763], Loss: 0.0062
LSTM Epoch [7/10], Step [2200/2763], Loss: 0.0934
LSTM Epoch [7/10], Step [2300/2763], Loss: 0.0945
LSTM Epoch [7/10], Step [2400/2763], Loss: 0.0039
LSTM Epoch [7/10], Step [2500/2763], Loss: 0.2947
LSTM Epoch [7/10], Step [2600/2763], Loss: 0.0071
LSTM Epoch [7/10], Step [2700/2763], Loss: 0.0037
LSTM Epoch [7/10], Validation Weighted Accuracy: 0.5000
LSTM Epoch [8/10], Step [100/2763], Loss: 0.0059
LSTM Epoch [8/10], Step [200/2763], Loss: 0.0028
LSTM Epoch [8/10], Step [300/2763], Loss: 0.0032
LSTM Epoch [8/10], Step [400/2763], Loss: 0.0057
LSTM Epoch [8/10], Step [500/2763], Loss: 0.0062
LSTM Epoch [8/10], Step [600/2763], Loss: 0.0944
LSTM Epoch [8/10], Step [700/2763], Loss: 0.0034
LSTM Epoch [8/10], Step [800/2763], Loss: 0.0069
LSTM Epoch [8/10], Step [900/2763], Loss: 0.0935
LSTM Epoch [8/10], Step [1000/2763], Loss: 0.2864
LSTM Epoch [8/10], Step [1100/2763], Loss: 0.0053
LSTM Epoch [8/10], Step [1200/2763], Loss: 0.0037
LSTM Epoch [8/10], Step [1300/2763], Loss: 0.0049
LSTM Epoch [8/10], Step [1400/2763], Loss: 0.0048
LSTM Epoch [8/10], Step [1500/2763], Loss: 0.0057
LSTM Epoch [8/10], Step [1600/2763], Loss: 0.2960
LSTM Epoch [8/10], Step [1700/2763], Loss: 0.0062
LSTM Epoch [8/10], Step [1800/2763], Loss: 0.0042
LSTM Epoch [8/10], Step [1900/2763], Loss: 0.0942
LSTM Epoch [8/10], Step [2000/2763], Loss: 0.0056
LSTM Epoch [8/10], Step [2100/2763], Loss: 0.0942
LSTM Epoch [8/10], Step [2200/2763], Loss: 0.0936
LSTM Epoch [8/10], Step [2300/2763], Loss: 0.0049
LSTM Epoch [8/10], Step [2400/2763], Loss: 0.0035
LSTM Epoch [8/10], Step [2500/2763], Loss: 0.0028
LSTM Epoch [8/10], Step [2600/2763], Loss: 0.2981
LSTM Epoch [8/10], Step [2700/2763], Loss: 0.0955
LSTM Epoch [8/10], Validation Weighted Accuracy: 0.5000

```
LSTM Epoch [9/10], Step [100/2763], Loss: 0.0042
LSTM Epoch [9/10], Step [200/2763], Loss: 0.2825
LSTM Epoch [9/10], Step [300/2763], Loss: 0.2956
LSTM Epoch [9/10], Step [400/2763], Loss: 0.0038
LSTM Epoch [9/10], Step [500/2763], Loss: 0.0045
LSTM Epoch [9/10], Step [600/2763], Loss: 0.0058
LSTM Epoch [9/10], Step [700/2763], Loss: 0.0043
LSTM Epoch [9/10], Step [800/2763], Loss: 0.2802
LSTM Epoch [9/10], Step [900/2763], Loss: 0.0056
LSTM Epoch [9/10], Step [1000/2763], Loss: 0.0060
LSTM Epoch [9/10], Step [1100/2763], Loss: 0.0963
LSTM Epoch [9/10], Step [1200/2763], Loss: 0.0039
LSTM Epoch [9/10], Step [1300/2763], Loss: 0.0042
LSTM Epoch [9/10], Step [1400/2763], Loss: 0.0057
LSTM Epoch [9/10], Step [1500/2763], Loss: 0.0031
LSTM Epoch [9/10], Step [1600/2763], Loss: 0.0944
LSTM Epoch [9/10], Step [1700/2763], Loss: 0.0941
LSTM Epoch [9/10], Step [1800/2763], Loss: 0.0943
LSTM Epoch [9/10], Step [1900/2763], Loss: 0.0907
LSTM Epoch [9/10], Step [2000/2763], Loss: 0.0052
LSTM Epoch [9/10], Step [2100/2763], Loss: 0.0058
LSTM Epoch [9/10], Step [2200/2763], Loss: 0.0940
LSTM Epoch [9/10], Step [2300/2763], Loss: 0.0038
LSTM Epoch [9/10], Step [2400/2763], Loss: 0.0046
LSTM Epoch [9/10], Step [2500/2763], Loss: 0.0956
LSTM Epoch [9/10], Step [2600/2763], Loss: 0.0949
LSTM Epoch [9/10], Step [2700/2763], Loss: 0.0067
LSTM Epoch [9/10], Validation Weighted Accuracy: 0.5000
LSTM Epoch [10/10], Step [100/2763], Loss: 0.0049
LSTM Epoch [10/10], Step [200/2763], Loss: 0.0071
LSTM Epoch [10/10], Step [300/2763], Loss: 0.0051
LSTM Epoch [10/10], Step [400/2763], Loss: 0.0045
LSTM Epoch [10/10], Step [500/2763], Loss: 0.0927
LSTM Epoch [10/10], Step [600/2763], Loss: 0.0042
LSTM Epoch [10/10], Step [700/2763], Loss: 0.0023
LSTM Epoch [10/10], Step [800/2763], Loss: 0.0954
LSTM Epoch [10/10], Step [900/2763], Loss: 0.0042
LSTM Epoch [10/10], Step [1000/2763], Loss: 0.0036
LSTM Epoch [10/10], Step [1100/2763], Loss: 0.0052
LSTM Epoch [10/10], Step [1200/2763], Loss: 0.0047
LSTM Epoch [10/10], Step [1300/2763], Loss: 0.0047
LSTM Epoch [10/10], Step [1400/2763], Loss: 0.0952
LSTM Epoch [10/10], Step [1500/2763], Loss: 0.0061
LSTM Epoch [10/10], Step [1600/2763], Loss: 0.0047
LSTM Epoch [10/10], Step [1700/2763], Loss: 0.0943
LSTM Epoch [10/10], Step [1800/2763], Loss: 0.0936
LSTM Epoch [10/10], Step [1900/2763], Loss: 0.0043
LSTM Epoch [10/10], Step [2000/2763], Loss: 0.0054
LSTM Epoch [10/10], Step [2100/2763], Loss: 0.0919
LSTM Epoch [10/10], Step [2200/2763], Loss: 0.0036
LSTM Epoch [10/10], Step [2300/2763], Loss: 0.0928
LSTM Epoch [10/10], Step [2400/2763], Loss: 0.3000
LSTM Epoch [10/10], Step [2500/2763], Loss: 0.0058
LSTM Epoch [10/10], Step [2600/2763], Loss: 0.0068
LSTM Epoch [10/10], Step [2700/2763], Loss: 0.0034
LSTM Epoch [10/10], Validation Weighted Accuracy: 0.5000
```

LSTM Weighted Accuracy on Test Set: 0.5000

Statement of Collaboration

It is mandatory to include a Statement of Collaboration in each submission, with respect to the guidelines below. Include the names of everyone involved in the discussions (especially in-person ones), and what was discussed.

All students are required to follow the academic honesty guidelines posted on the course website. For programming assignments, in particular, I encourage the students to organize (perhaps using Campuswire) to discuss the task descriptions, requirements, bugs in my code, and the relevant technical content before they start working on it.

However, you should not discuss the specific solutions, and, as a guiding principle, you are not allowed to take anything written or drawn away from these discussions (i.e. no photographs of the blackboard, written notes, referring to Campuswire, etc.).

Especially after you have started working on the assignment, try to restrict the discussion to Campuswire as much as possible, so that there is no doubt as to the extent of your collaboration.

Complete your statement of collaboration here:

I did not ask nor had any questions from TA or my peers while I was working on this assignment.

What to submit

- Export a notebook as PDF
 - Go to Main menu | File and select Print . pdf.
- Upload your jupyter notebook PDF on gradescope
- The notebook must have output values for the final test accuracy.
- Do not submit the data file or directories.